



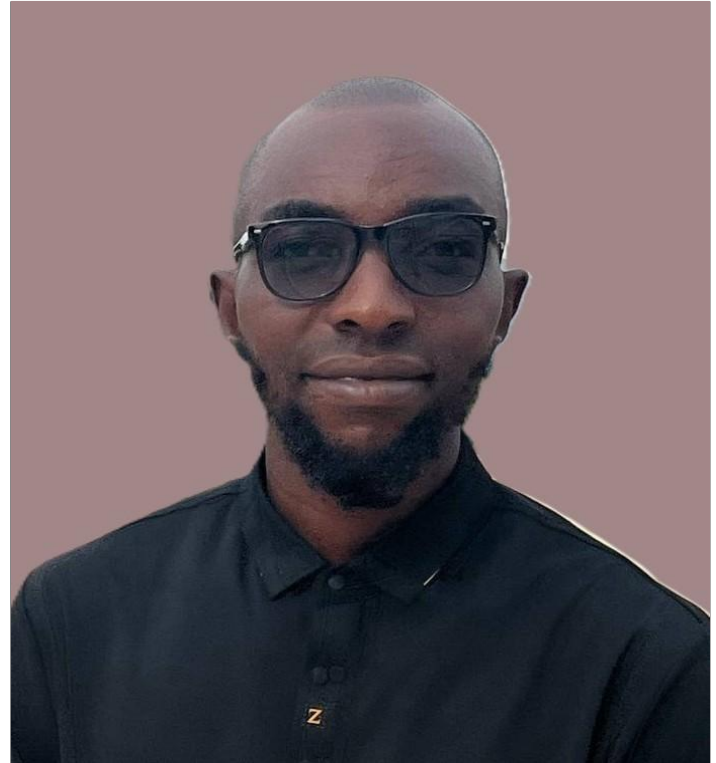
Lisk Summer Bootcamp

Etette Etok

Software Engineer & Web3 DevRel

Twitter: @Em_Etok

Linkedin: Etette Etok



Today's Agenda

- What is Solidity?
- Solidity file layout
- Solidity data types
- Solidity functions
- Solidity storage



WHAT IS SOLIDITY

Solidity is a high-level, object-oriented programming language for Ethereum smart contracts. It is statically typed, supports inheritance, libraries and complex user-defined types. Smart contracts are programs that govern the behavior of accounts within the Ethereum state.

Low level solidity compiles human readable code into bytecodes that translate to [OPCODES](#) on the [EVM](#).

The Ethereum Virtual Machine (EVM) is a decentralized virtual environment that executes code consistently and securely across all Ethereum nodes, using **gas** to measure the **computational effort** required for **operations** ensuring efficient resource allocation and network security.

Why Solidity?

- Powers decentralized applications by smart contract logics execution on Ethereum Virtual Machine.
- Enables trustless, secure and transparent transactions.
- Trusted by millions of blockchain developers

SOLIDITY FILE LAYOUT

- [SPDX License Identifier](#): Indicates the license for the source code (MIT, UNLICENSED, UNLICENSE etc), enhancing trust by making code licensing clear.
- Pragma: The pragma keyword is used to enable certain compiler features or checks and are local to each source file.
- Imports:
 - `import "filename";`
 - `import * as alias from "filename";`
 - `import {symbol1 as alias, symbol2} from "filename";`
- Comments:
 - Single-line: `// Comment`
 - Multi-line: `/* Comment */`
- Contract Definition: Is the core of a solidity file, containing contract declaration, structs, enums, functions, etc.

Layout of a Solidity Source File



SOLIDITY VARIABLES & CONDITIONALS

Variables

There are 3 types of variables in Solidity

- Local variable: declared inside a function but not stored on the blockchain
- State variable: declared outside a function and stored on the blockchain
- Global variable: provides information about the blockchain.

Control Structure

Most of the control structures known from curly-braces languages are available in Solidity. If, else if, else, while, do, for, break, continue & return semantics are available in solidity to control and structure the flow and execution of logic. Additionally, logic can be controlled with the revert() or require() special functions.

SOLIDITY DATA TYPES

Solidity is a statically typed language, which means that the type of each variable needs to be specified. Solidity also provides several elementary types (Value Types) which can be combined to form complex types (Reference Types).

Value Type

- **Bool**: true or false values for conditions.
- **Integer**: signed and unsigned integers of various sizes.
- **String**: UTF-8 strings for text data.
- **Bytes**: hold a sequence of bytes from 1 to up to 32.
- **Address**: 20-byte Ethereum addresses for EOA accounts or contracts/smart accounts.

Reference Type

- **Data Location**: to explicitly provide the data area where the data type is stored. Memory, Storage, Calldata
- **Array**: can be fixed size or dynamic size. String[], string[5]
- **Struct**: custom structures to group multiple data types.
- **Mapping**: Key value pairing of same or different data types
- **Enum**: named constants for categorical data.

SOLIDITY FUNCTIONS

Functions are the executable units of code, usually defined inside a contract but they can also be defined outside of contracts. A solidity function accepts typed parameters as input and may return an arbitrary number of values as output.

Function Components:

- **Visibility**: Determines who can call the function (public, private, internal, external).
- **Mutability**: Indicates if the function modifies the Ethereum state (pure, view, payable).
- **Parameters**: Inputs to the function defined with types.
- **Return Values**: Outputs from the function with specified types.
- **Modifiers**: Custom logic to run before the function (access control, input validation etc) executes.

Special Functions:

- **Receive()**: enables a smart contract to receive ETH
- **Fallback()**: is executed either when a function that does not exist is called or ETH is sent to a contract with no receive() function or msg.data is not empty.

SOLIDITY STORAGE

Storage in Solidity refers to the persistent data that is stored on the blockchain. It is where state variables are kept and can be accessed and modified by functions.

Storage Components:

- State Variables: Variables declared at the contract level, stored in storage. Here data is stored contiguously item after item starting with the first state variable.
- Arrays: Dynamic or fixed-size arrays stored in storage.
- Mappings: Key-value pairs that are also stored in storage.
- Structs: Custom structures that can contain multiple variables, stored in storage

The End

Questions?

ASSIGNMENT

Create a smart contract that allows users to register and update their profile information (e.g., name, age etc).

Functions: Create register(), updateProfile(), getProfile()

Requirements:

- Use a struct named User
- Store data in state variables
- Include public getter function
- Validate that user cannot register twice
- Add a view function to fetch user info
- Track registration timestamp using uint

Submission:

Push your smart contract to github and submit a link to the github repo